Keresés hasító táblázatokban. Láncolt listás és közvetlen címzéses hasító technika.

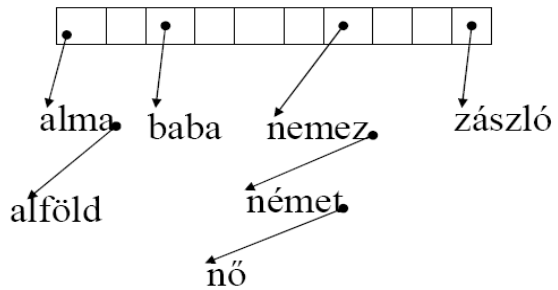
Hasító (hash) táblázatok
Hasítótáblák

Adott egy nagyméretű univerzum amelyhez a kulcsuk által azonosított elemeket tartoznak. Ezek közül szeretnénk elemeket tárolni egy m méretű T tömbben úgy, hogy az elemek a kulcs alapján hatékonyan megtalálhatóak legyenek. A továbbiakban feltesszük, hogy az elemek maguk a kulcsok, de a valódi alkalmazásokban a kulcsok csak azonosításra szolgálnak, és az elemek további adatokat tartalmaznak.

Választunk egy $h : U \rightarrow \{0, \ldots, m-1\}$ hasítófüggvényt, amely minden a halmazelemre megadja azt a tömbindexet, ahol az elemet tárolni szeretnénk $T[h(a)] := a$.

Abban az esetben van probléma, ha két különböző elemet ugyanott szeretnénk tárolni azaz, ha $a = b$ és $h(a) = h(b)$. Az ilyen esetekben ütközésről beszélünk. Az ütközések feloldására két elterjedt módszer a láncolás és a nyílt címzés.
Ütközésfeloldás láncolással
Ebben az esetben az ütközést úgy oldjuk fel, hogy ugyanarra a helyre rakjuk az elemeket. Tehát a T tömb elemei láncok, minden láncelem egy adatból és egy csat mutatóból áll, amely a következő láncelemre mutat.

Ütközésfeloldás láncolással



- Alapötlet: az ütköző (ugyanazon címre leképeződő) elemeket láncolt listába összefogjuk
- Beszúr(x)
  hash=h(kulcs(x))
  if T(hash)<>NIL then T(hash).Beszúr(x)
  else T(hash)=LáncLista(x)
- Keres(x)
  hash=h(kulcs(x))
  if T(hash)=NIL then return NIL
  else return(T(hash).Keres(x))
- Töröl(x)
  hash=h(kulcs(x))
  if T(hash)<>NIL then T(hash).Töröl
  if T(hash).Méret=0 then T(hash)=NIL

Láncolásos hasító technika elemzése

- Legrosszabb eset: egyetlen láncolt lista
- Kitöltöttségi arány (a=n/m), ahol n a tárolt elemek összes száma, m a hash-tábla hossza
- Tétel: láncolt hasító technika és egyenletes hash függvény esetén, az átlagos keresési idő: $\Theta(1+a)$.
- Biz: az átlagos keresési idő az átlagos lista végigjárásával arányos (sikeres és sikertelen keresés)

• Következtetés: 1. Fix hash-tábla esetén, sok elemre lineáris Θ(n) 2. Ha a hash-tábla a tárolt elemekkel
arányosan nő, ill. kevés a tárolt elem, akkor Θ(1)

Hasító függvények

• Mikor jó a hasító függvény? Ha a láncok kb. egyenletesen nőnek<=azaz a T(y) index egyenletes
• Mivel T(y)=T(h(U(x))), függ az U(x) kulcseloszlástól
• Milyen az U(x) eloszlása?
- fordítóprogramok: egymáshoz közeli szimbólumok
• H eloszlása független legyen az adatok esetleges szabályszerűségeitől
- pl. a kulcs maradéka egy prímszámra nézve
• Feltételezzük, hogy a kulcs egész (ha nem az, akkor leképezhető)

Hasító függvények

• Osztásos módszer: h(k)=k mod m, ahol m a hash tábla mérete
• Szorzásos módszer h(k)=m*(k*A mod 1)), ahol A egy állandó, mod 1: törtrészképzés
  Pl. tfh: k belefér egy gépi szóba, és m=2p. Ekkor k*(A*m) egy gépi szorzás → 2 szó, ez mod m az alsó p bit levágását jelenti.MFC (Donald Knuth)
• Minden rögzített hasító függvényhez létezik
  „rossz" adat (csak 1 lánc jön létre, Θ(n) elérés) → a tényleges függvényt adott függvényosztályból véletlenszerűen (és adatfüggetlenül) választjuk ki. „Univerzális hasítási technika"

Nyílt címzés

• Nyílt címzésű a hash tábla, ha az adatok NEM láncolt listában, hanem benn a táblában vannak tárolva. Ütközés esetén újabb és újabb pozíciókat próbálunk ki, amíg csak üresre nem bukkanunk
  → a tábla betelhet
  → nincs mutató, nincs láncolt lista
  → a kipróbálandó rések címét a hash függvény (a kipróbálási számtól is függően) adja meg
  → adott kulcshoz a hash függvény a T címtér egy permutációját adja meg (vagyis mindent kipróbál)

Ütközésfeloldás nyílt címzéssel

(más: Nyílt címzés esetén, egy helyre csak egy elemet teszünk, és az ütközést úgy oldjuk fel, hogy nem csak egy hasítófüggvényünk van, hanem minden elemhez tartozik egy próbasorozat. Az elemet a próbasorozatból az első üres helyre tesszük.)

A nyílt címzéses hash technika elemzése

• Kitöltöttségi arány (a=n/m), ahol n a tárolt elemek összes száma, m a hash-tábla hossza → a<=1
• Tfh. A hasítás egyenletes, vagyis egy (h(k,0), h(k,1),…h(k,m-1)) kipróbálási sorozat egyenletesen állítja elő a (0,1,…,m) sorozat permutációit.
• Legyen pi annak valószínűsége, hogy pontosan i próba talál foglalt rést. (0<=i<=n-1), i>n-re pi=0
• A próbák számának várhatóértéke tehát: 1+i=0Σ∞i*pi
• Legyen qi annak valószínűsége, hogy legfeljebb i próba talál foglalt rést. (aztán jön a szabad)
• i=0Σ∞i*pi = i=0Σ∞qi (annak a valószínűsége, hogy pontosan 0;1;… próba talál rést, ugyanannyi, mint a valószínűsége., hogy legfeljebb 0;1;…próba talál rést
• Mennyi a qi-k értéke?
• q1=n/m annak a valószínűsége., hogy az első próba foglalt elemet talál
• q2=n/m*(n-1)/(m-1)
• qi=n/m*(n-1)/(m-1)*…*(n-i+1)/(m-i+1)<= ai
• A próbák számának várhatóértéke: 1+i=0Σ∞i*pi=1+i=0Σ∞qi <=1+a1 +a2 +a3 +...=1/(1-a)

• 1. Elem biztos, 2. Elem a, 3. Elem a2...
• Ha félig van kitöltve, akkor próbaszám=2 lépés
• Ha 90%-ig van kitöltve, akkor próbaszám=10 lépés

• Kitöltöttségi arány (a=n/m), ahol n a tárolt elemek összes száma, m a hash-tábla hossza → a<=1
• Ilyenkor a sikeres keresés várható próbaszáma: 1/a*ln(1/(1- a))
• Bizonyítás nélkül… (az előzőhöz hasonló)
    → 50% kitöltöttségre: 1,387
    → 90% kitöltöttségre: 2.559
    → a nyílt címzéses hash technika a keresés optimalizálását a beszúrás lassulásán keresztül éri el…